

CHARTBOT AI – Intelligent Database Chat Agent

Project Report

Abstract

ChartBot AI is an intelligent database chat application that enables users to interact with a database using natural language instead of SQL queries. The system converts user questions into SQL queries using a locally running Large Language Model (LLM) through Ollama. It retrieves data from the database, presents it in tabular form, generates interactive charts, and provides explanations of the results. The application also supports voice input, ER diagram visualization, and process flow diagrams, making database analysis easier for both technical and non-technical users.

1. Introduction

Organizations store large amounts of data in databases, but retrieving meaningful information usually requires SQL knowledge. Many users find it difficult to write SQL queries or analyze database structures.

ChartBot AI addresses this problem by allowing users to communicate with the database using plain English. The application automatically understands the user's request, generates SQL queries, executes them safely, visualizes the results, and explains the findings.

The project is completely offline because it uses Ollama to run the language model locally without depending on cloud APIs.

2. Problem Statement

Traditional database management systems require users to understand SQL syntax for retrieving information. This creates a barrier for beginners and non-technical users.

Challenges include:

- Requirement of SQL knowledge
- Difficulty understanding database relationships
- Lack of interactive visualization
- Time-consuming manual query writing
- Limited accessibility for beginners

3. Objectives

The objectives of ChartBot AI are:

- Convert natural language into SQL queries.
- Execute only safe SELECT queries.
- Display results in an interactive table.
- Generate charts automatically.
- Support voice-based database queries.
- Display ER diagrams and process flow diagrams.
- Provide explanations of query results.
- Operate completely offline using Ollama.

4. Proposed System

The proposed system consists of a Streamlit-based web interface connected to an SQLite database. User questions are processed by a locally running Qwen 2.5-Coder model through Ollama.

The workflow is:

User Question



Natural Language Processing



SQL Query Generation



Database Execution



Data Retrieval



Visualization & Insights



User Response

5. System Architecture

Components include:

- Frontend
 - Streamlit
 - Interactive Chat Interface
 - Voice Input
 - Charts
 - ER Diagram Viewer
- Backend
 - Python
 - SQLite Database
 - Ollama API
 - Query Execution Engine
- AI Model
 - Qwen2.5-Coder 3B running locally through Ollama

6. Technologies Used

Technology	Purpose
Python	Backend development
Streamlit	User Interface
SQLite	Database
Ollama	Local LLM execution
Qwen2.5-Coder 3B	SQL generation
Pandas	Data processing
Plotly	Interactive charts
Mermaid	ER diagrams & flowcharts
SpeechRecognition	Voice input
Docker	Containerized deployment
GitHub	Version control

7. Modules

User Interface

Provides a modern chat interface where users ask questions in natural language.

Features

- Chat window
- Voice input
- Quick action buttons
- SQL viewer
- Data tables
- Interactive charts

AI Query Generator

Processes natural language questions and converts them into SQL queries using the Qwen model running through Ollama.

Database Engine

Executes generated SQL queries on the SQLite database while restricting operations to safe SELECT statements.

Data Visualization

Automatically creates:

- Bar Charts
- Line Charts
- Pie Charts
- Scatter Plots

using Plotly.

Diagram Generator

Displays:

- Entity Relationship Diagram
- Process Flow Diagram

using Mermaid.

Voice Assistant

Uses Speech Recognition to convert spoken questions into text before processing.

8. Features

- Natural Language Database Querying
- SQL Generation
- Interactive Charts
- Voice Input
- ER Diagram Visualization
- Process Flow Diagram
- Local AI using Ollama
- Secure Query Execution
- Data Insights
- User-Friendly Interface

9. Workflow

1. User enters a question.
2. The application sends it to Ollama.
3. Ollama generates SQL.
4. SQL is validated.
5. Query executes on SQLite.
6. Results are displayed.
7. Charts are generated.
8. Insights are shown.
9. Conversation history is maintained.

10. Advantages

- No SQL knowledge required
- Faster database querying
- Interactive visualization
- Offline operation
- Voice support

- Beginner friendly
- Secure SELECT-only execution
- Easy deployment using Docker

11. Future Enhancements

- MySQL support
- PostgreSQL support
- SQL Server support
- User authentication
- Multi-user access
- PDF report generation
- Export to Excel
- Dashboard analytics
- Improved AI accuracy
- Cloud deployment

12. Testing

The application was tested using different natural language queries such as:

- Show top 5 products by revenue
- Which city has most customers?
- Show monthly order trends
- What is total revenue?
- Show ER diagram
- What is machine learning?

The generated SQL queries, retrieved data, and visualizations were verified for correctness.

13. Deployment

The application is designed for deployment using Docker.

Deployment components:

- Streamlit Container
- Ollama Container
- SQLite Database
- Docker Compose

This enables the application to run consistently across different operating systems.

14. Conclusion

ChartBot AI simplifies database interaction by allowing users to communicate using natural language. By combining Streamlit, SQLite, Plotly, Speech Recognition, and the Qwen 2.5-Coder language model through Ollama, the system provides intelligent querying, visualization, and explanation of database information. The application demonstrates how local AI models can improve accessibility while maintaining privacy through offline execution.

References

1. Python Documentation – <https://docs.python.org/>
2. Streamlit Documentation – <https://streamlit.io/>
3. SQLite Documentation – <https://sqlite.org/>
4. Ollama Documentation – <https://ollama.com/>
5. Plotly Documentation – <https://plotly.com/python/>
6. Pandas Documentation – <https://pandas.pydata.org/>
7. Mermaid Documentation – <https://mermaid.js.org/>
8. Docker Documentation – <https://docs.docker.com/>
9. GitHub Documentation – <https://docs.github.com/>